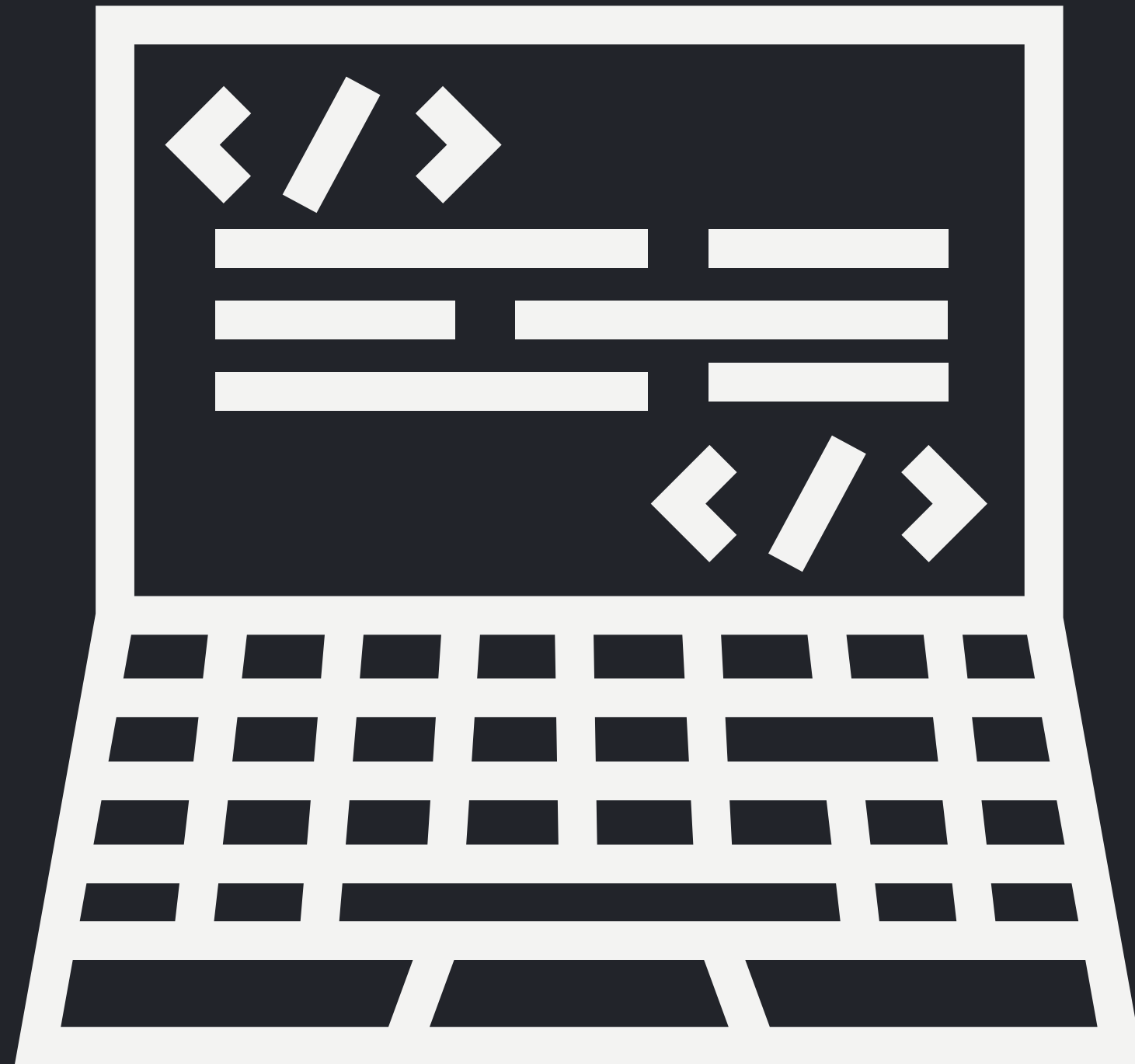


Design Patterns



Sistema de Biblioteca: Evolução Arquitetural

Do MVP acoplado a uma arquitetura modular extensível

Jefferson Barzan Alexandrino

Problema Escolhido

Problema

- Bibliotecas precisam controlar:
- Acervo físico
- Empréstimos
- Devoluções
- Estoque
- Integrações futuras com novos tipos de mídia

Desafio

Criar um sistema que inicialmente resolvesse o problema básico e depois pudesse evoluir sem aumentar a complexidade do código.



Sobre o sistema

Objetivo

- Gerenciar livros físicos e digitais
- Controlar empréstimos, vendas e devoluções
- Manter estoque atualizado automaticamente

Funcionalidades

- Cadastro de livros
- Empréstimos
- Venda
- Devoluções
- Integração com E-books externos

Tecnologias

Frontend

- HTML: Estrutura das páginas
- CSS: Estilização, cores, layouts
- JavaScript (Client-side): Validações, manipulação do DOM e requisições assíncronas (fetch/axios) ao backend.

Backend

- Node.js: O ambiente de execução que roda o JavaScript no servidor.
- Express: O framework que gerencia as rotas , requisições (GET, POST) e respostas do servidor.

Banco de Dados

- SQLite: armazenado em um único arquivo local

O Ponto de Partida: MVP Funcional, mas Rígido



Estrutura Original

Todas as responsabilidades concentradas em um único arquivo `server.js` na raiz: inicialização, tabelas, conexão SQLite, rotas Express e queries SQL brutas.

Problemas Arquiteturais Identificados

- **Baixa coesão:** múltiplas responsabilidades misturadas no mesmo escopo
- **Alto acoplamento:** rotas HTTP diretamente ligadas às tabelas físicas do banco
- **Violação do SRP:** o módulo de rede também gerencia persistência e lógica de negócio

A Necessidade de Evolução

Após o protótipo funcional, novos requisitos emergiram, exigindo extensibilidade sem reescrever o núcleo do sistema.

1

Integração com API Externa

Incorporação instantânea de E-books de um parceiro externo, com dados em formato incompatível com o modelo local.

2

Automação de Tarefas Colaterais

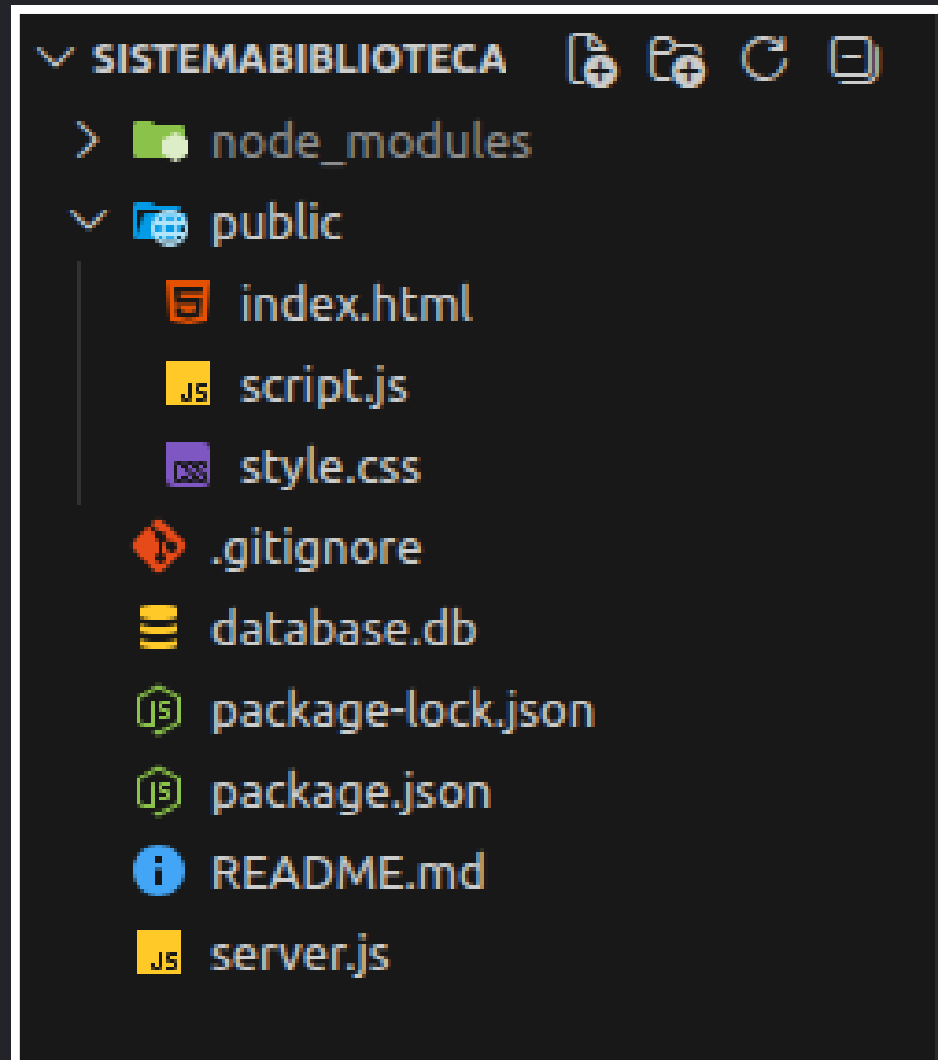
Após cada empréstimo: baixa automática no estoque físico e envio (ou simulação) de e-mail de notificação ao usuário.

3

Prevenção de Concorrência no SQLite

Com múltiplos módulos acessando o arquivo .db, surgiram travamentos do tipo "database is locked" que precisavam ser eliminados.

Demonstração Funcional: Versão 1.0 vs 2.0



VÍDEO: VERSÃO 1.0

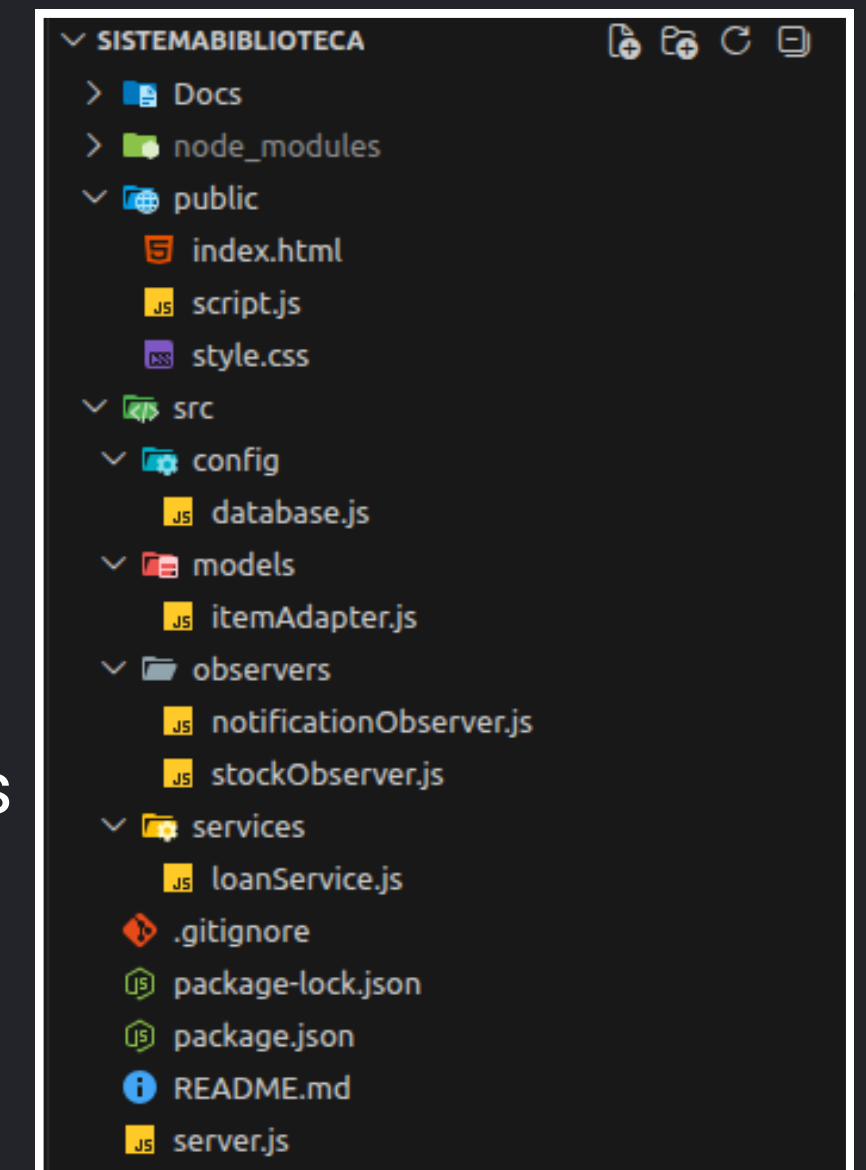
Interface Legada

Listas simples, fluxo estrito e limitado a dados locais. Sem integração externa, sem logs de eventos. Sem css e frontend muito elaborado.

VÍDEO: VERSÃO 2.0

Interface Refatorada

Interface moderna em tabelas exibindo dinamicamente E-books externos integrados via API. Terminal registra disparos automáticos de estoque e notificações em tempo real.

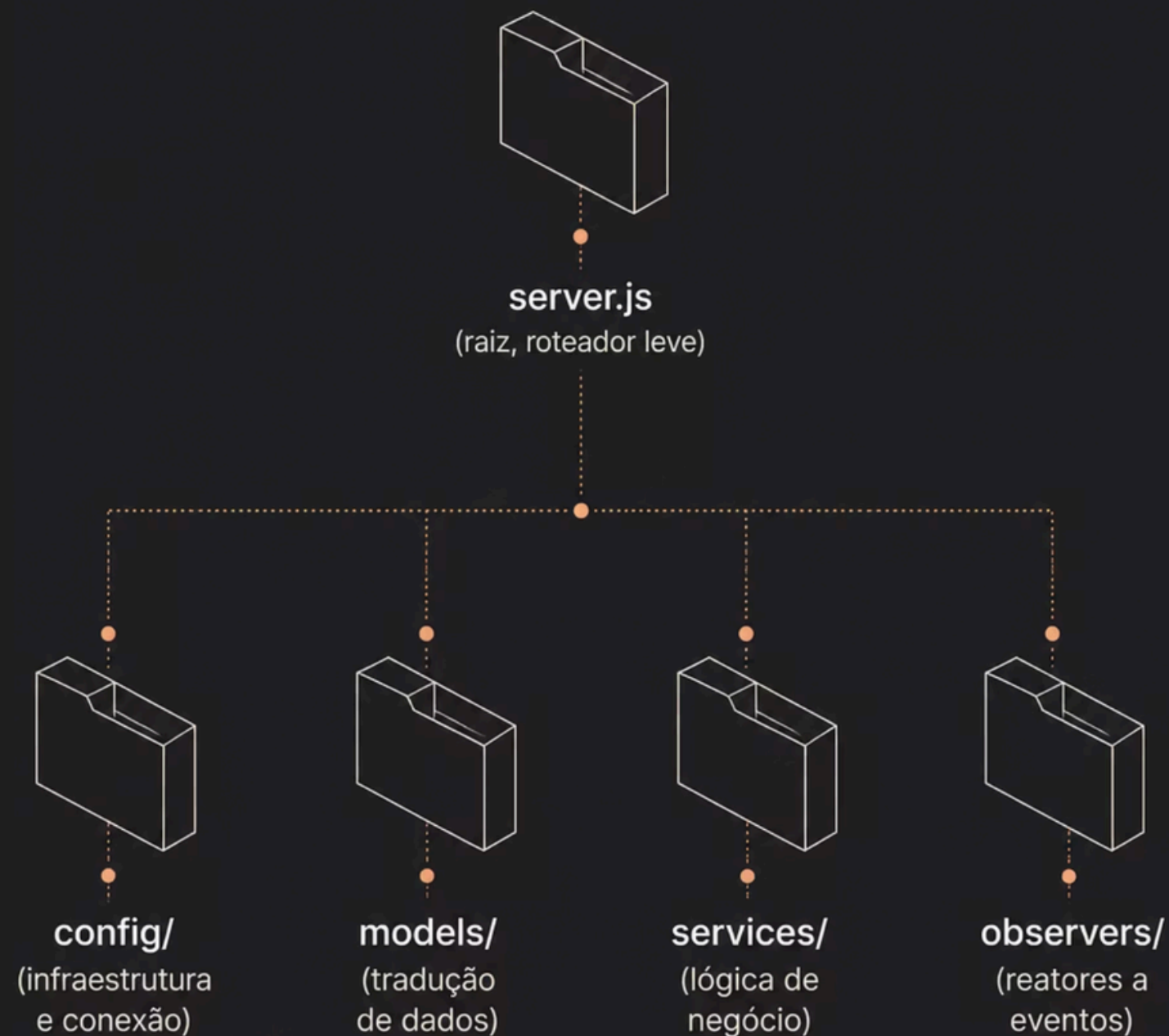


A Nova Estrutura de Pastas e Desacoplamento

De monolito a orquestrador

O server.js foi limpo e transformado em um roteador leve na raiz. O núcleo técnico foi delegado para pacotes especializados dentro de src/:

- **src/config/** — Centralização da infraestrutura e conexão com banco
- **src/models/** — Camada de tradução e normalização de dados
- **src/services/** — Lógica de negócio encapsulada por domínio
- **src/observers/** — Reatores independentes a eventos publicados



Singleton no Gerenciamento do SQLite



Onde foi aplicado

`src/config/database.js`

Problema Real Tratado

Com a criação de múltiplos serviços de negócio, o risco de múltiplas conexões concorrentes bloqueando o arquivo físico .db crescia a cada novo módulo adicionado.

Solução Técnica

Classe de controle que garante **uma única instância de conexão** aberta e imutável (`Object.freeze`), permitindo reaproveitamento seguro em toda a aplicação sem conflitos de acesso.

Singleton no Gerenciamento do SQLite

```
src > config > js database.js > Database > constructor
1  const sqlite3 = require("sqlite3").verbose();
2  const path = require("path");
3
4  class Database {
5      constructor() {
6          if (!Database.instance) {
7              const dbPath = path.resolve(__dirname, "../..../database.db");
8              console.log("Banco de dados em:", dbPath);
9
10             this.connection = new sqlite3.Database(dbPath, (err) => {
11                 if (err) {
12                     console.error("Erro ao conectar ao banco:", err.message);
13                 } else {
14                     console.log("Banco de dados conectado com sucesso!");
15                 }
16             });
17
18             this.init();
19             Database.instance = this;
20         }
21         return Database.instance;
22     }
23 }
```

```
4  init() {
5      this.connection.serialize(() => {
6          this.connection.run(`
7              CREATE TABLE IF NOT EXISTS livros (
8                  id INTEGER PRIMARY KEY AUTOINCREMENT,
9                  titulo TEXT NOT NULL,
10                 autor TEXT NOT NULL,
11                 ano INTEGER NOT NULL,
12                 quantidade INTEGER NOT NULL DEFAULT 0,
13                 disponivel_venda INTEGER DEFAULT 0
14             )
15         `, (err) => {
16             if (err) console.error("Erro ao criar tabela livros:", err);
17             else console.log("Tabela 'livros' pronta");
18         });
19
20         this.connection.run(`
21             CREATE TABLE IF NOT EXISTS emprestimos (
22                 id INTEGER PRIMARY KEY AUTOINCREMENT,
23                 usuario TEXT NOT NULL,
24                 livro_id INTEGER NOT NULL,
25                 status TEXT NOT NULL,
26                 data_criacao DATETIME DEFAULT CURRENT_TIMESTAMP
27             )
28         `, (err) => {
29             if (err) console.error("Erro ao criar tabela emprestimos:", err);
30             else console.log("Tabela 'emprestimos' pronta");
31         });
32     });
33 }
34
35 getConnection() {
36     return this.connection;
37 }
38
39 const instance = new Database();
40 Object.freeze(instance);
41
42 module.exports = instance;
```

- Validação da instância única (Singleton)
 - Se a instância já existir, ela é reutilizada.
- Criação da conexão com o banco
 - Caso não exista instância, é aberta uma nova conexão com o SQLite.
- Garantia de imutabilidade
 - Uso do `Object.freeze()` para impedir alterações no objeto exportado.
- Compartilhamento da conexão
 - Toda a aplicação utiliza o mesmo canal de comunicação com o banco de dados.
- Benefício da abordagem
 - Maior segurança, consistência e controle da infraestrutura de acesso aos dados.

Adapter na Integração com API de E-books

Onde foi aplicado

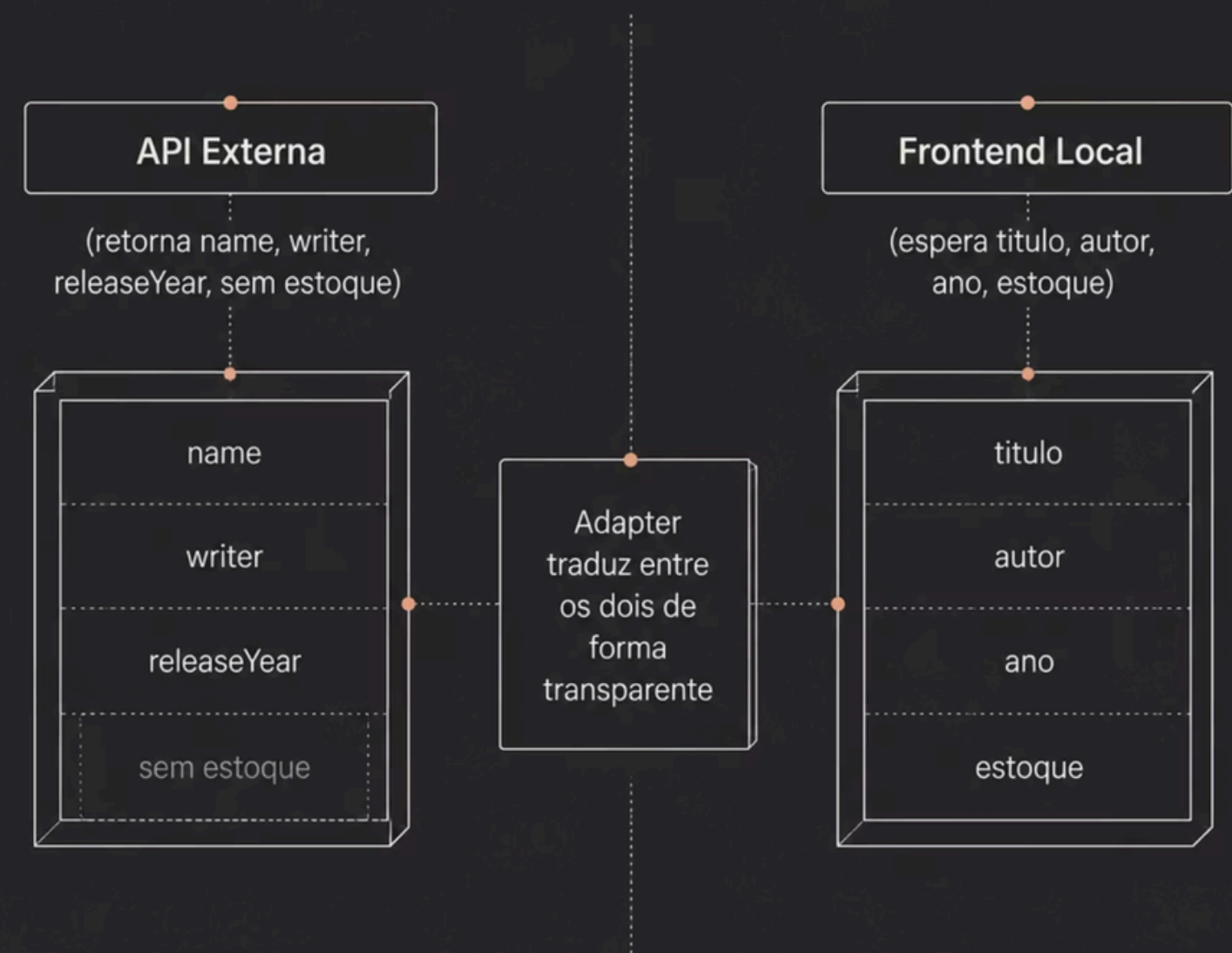
`src/models/itemAdapter.js`

Problema Real Tratado

A API externa retornava dados em inglês (name, writer, releaseYear) sem campo de estoque, enquanto o frontend dependia de .titulo, .autor e .ano.

Solução Técnica

Classe adaptadora que traduz objetos estrangeiros em tempo de execução. O frontend consome dados de forma **transparente e unificada**, eliminando condicionais if/else espalhados pelo código.



Adapter na Integração com API de E-books

```
src > models > JS itemAdapter.js > ...
1  class ExternalEBook {
2      constructor(id, name, writer, releaseYear) {
3          this.id = id;
4          this.name = name;
5          this.writer = writer;
6          this.releaseYear = releaseYear;
7      }
8  }
9
10 class EBookAdapter {
11     constructor(externalEBook) {
12         this.id = 9000 + externalEBook.id;
13         this.titulo = `[E-Book] ${externalEBook.name}`;
14         this.autor = externalEBook.writer;
15         this.ano = externalEBook.releaseYear;
16         this.quantidade = 999;
17     }
18 }
19
20 module.exports = { ExternalEBook, EBookAdapter };
```

- Classe EBookAdapter
 - Recebe no construtor o objeto retornado pela API externa.
- Tradução dos dados
 - Mapeia propriedades em inglês para os atributos utilizados pelo sistema.
- Compatibilidade com a aplicação
 - Mantém a mesma estrutura esperada pelas interfaces e rotas existentes.
- Tratamento de mídias digitais
 - Atribui um ID fictício e estoque de 999.
- Regra de negócio aplicada
 - Representa que e-books não possuem limitação de estoque físico.
- Benefício da abordagem
 - Permite que livros físicos e digitais sejam tratados de forma unificada.
- Resultado no frontend
 - Todos os itens são exibidos na mesma tabela, sem necessidade de condicionais complexas (if/else).

Observer para Desacoplamento de Eventos

Onde foi aplicado

`src/services/loanService.js` (Subject) e pasta `src/observers/`

Problema Real Tratado

Evitar que a rota HTTP de empréstimo acumulasse códigos de e-mail e manipulação direta de inventário em uma função gigante.

Solução Técnica

O empréstimo tornou-se um **publicador de eventos**. StockObserver e NotificationObserver reagem de forma independente. Aplicação direta do **OCP**: novas automações futuras podem ser plugadas sem alterar a rota principal.



Observer para Desacoplamento de Eventos

```
src > services > JS loanService.js > ...
1  class LoanService {
2    constructor() {
3      this.observers = [];
4    }
5
6    subscribe(observer) {
7      this.observers.push(observer);
8    }
9
10   async notify(event, data) {
11     for (const observer of this.observers) {
12       await observer.update(event, data);
13     }
14   }
15 }
16
17 module.exports = LoanService;
```

```
1  const express = require("express");
2  const path = require("path");
3  const dbInstance = require("../src/config/database");
4  const LoanService = require("../src/services/loanService");
5  const StockObserver = require("../src/observers/stockObserver");
6  const NotificationObserver = require("../src/observers/notificationObserver");
7  const { ExternalEBook, EBookAdapter } = require("../src/models/externalEBook");
8
9  const app = express();
10 const PORT = 3000;
11
12 app.use(express.json());
13 app.use(express.static("public"));
14
15 // Configura o Observer
16 const loanService = new LoanService();
17 loanService.subscribe(new StockObserver());
18 loanService.subscribe(new NotificationObserver());
19
```

- Implementação do padrão Observer
 - O loanService.js concentra a lógica de inscrição e notificação dos observadores.
- Métodos principais
 - subscribe: registra novos observadores.
 - notify: comunica os observadores de forma assíncrona.
- Integração na inicialização
 - O server.js conecta os componentes do sistema.
- Registro dos observadores
 - Observador de estoque.
 - Observador de notificações.
- Facilidade de expansão
 - Novas funcionalidades podem ser adicionadas criando novos observadores.
- Exemplo de evolução
 - Um sistema de auditoria exigiria apenas uma nova classe e um novo subscribe.
- Benefício arquitetural
 - A rota de empréstimo permanece inalterada.
- Princípio aplicado
 - Demonstra o Princípio Aberto/Fechado (Open/Closed Principle), permitindo extensão sem modificar o código existente.

Comparação Final

MVP	REFATORADO
Tudo no server.js	Estrutura modular
Alto acoplamento	Baixo acoplamento
Difícil expandir	Fácil expandir
Sem API externa	Adapter
Sem eventos	Observer
Risco de múltiplas conexões	Singleton

Conclusão: Impactos da Evolução de Design

A refatoração orientada a padrões produziu ganhos mensuráveis na qualidade de engenharia do sistema.

Alta Coesão

Cada módulo tem uma única razão para mudar.
Responsabilidades claramente delimitadas por domínio.

Baixo Acoplamento

Camadas de rede, dados e negócio operam de forma independente, comunicando-se via interfaces estáveis.

Extensibilidade (OCP)

Novos módulos plugáveis sem modificar código existente. Observer elimina condicionais de crescimento.

Resiliência e Escalabilidade

Singleton previne conflitos de concorrência no SQLite.
Sistema pronto para novos requisitos sem débito técnico.

✅ Sistema 100% aderente aos princípios SOLID — SRP e OCP comprovados na prática.